Inventory Pro

Système de Gestion d'Inventaire Professionnel

RAPPORT TECHNIQUE COMPLET

Projet de Fin d'Études (PFE)

Type : Application Web Full-Stack

Frontend : Angular 21 • Backend : ASP.NET Core

Base de données : SQL Server

Date : Avril 2026

📑 Table des Matières

1. Présentation Générale de l'Application
2. Ce que fait l'Application — Fonctionnalités Détaillées
3. Architecture Technique
4. Stack Technologique
5. Modules Fonctionnels (Frontend)
6. Architecture Backend (API REST)
7. Modèle de Données
8. Système d'Authentification & Autorisation
9. Matrice des Permissions
10. Services & Logique Métier
11. Fonctionnalités Temps Réel (SignalR)
12. APIs REST — Contrôleurs
13. Sécurité
14. Interface Utilisateur (UI/UX)
15. Déploiement & Configuration
16. Résumé des Statistiques du Projet
17. Mises à Jour Récentes et Améliorations

1. Présentation Générale de l'Application

Inventory Pro est une application web full-stack de gestion d'inventaire conçue pour permettre aux entreprises de suivre leurs produits, stocks, mouvements d'entrée/sortie, sites d'entreposage et alertes

de stock en temps réel. L'application est développée dans le cadre d'un **Projet de Fin d'Études (PFE)**.

Objectifs principaux

- Gestion complète des produits (CRUD, catégorisation, code-barres, images)
- Suivi des stocks par site avec seuils d'alerte configurables
- Enregistrement et traçabilité des mouvements de stock (entrées/sorties)
- Gestion multi-sites (entrepôts, magasins, centres de distribution)
- Système d'alertes automatiques basé sur les seuils de stock
- Scanner de codes-barres intégré (caméra ou QR code externe)
- Gestion des utilisateurs avec contrôle d'accès basé sur les rôles (RBAC)
- Suggestions de réapprovisionnement basées sur les seuils
- Tableau de bord (Dashboard) avec statistiques en temps réel

2. Ce que fait l'Application — Fonctionnalités Détaillées

Cette section décrit en détail **ce que l'application permet de faire** du point de vue de l'utilisateur final, en couvrant chaque fonctionnalité majeure avec ses cas d'utilisation concrets.

📊 2.1 Tableau de Bord (Dashboard)

Le Dashboard est la page **Aperçu Réapprovisionnement** de l'application. Il agrège les données produits/stocks/mouvements/alertes et se met à jour automatiquement toutes les 30 secondes, avec un bouton de rafraîchissement manuel et un accès rapide vers `/movements`.

Filtres Disponibles

Filtre	Valeurs	Effet
Type de site	Tous confondus, Entrepôts, Magasins	Restreint la portée d'analyse à un type d'emplacement
Emplacement	Tous les sites ou site spécifique	Filtre les KPI, graphes et tableaux au site sélectionné
Catégorie	Toutes les catégories ou catégorie ciblée	Filtre les produits et stocks associés
Produit	Tous les produits ou produit ciblé	Zoom sur un produit précis
Période d'analyse	Aujourd'hui, 7 jours, 30 jours, ce mois, tout l'historique	Filtre les mouvements et alertes par intervalle temporel

Cartes KPI réellement affichées

KPI	Description
Valorisation du Stock	Somme monétaire calculée à partir du stock disponible et du prix produit
Unités en Stock	Quantité totale en stock selon les filtres actifs
Produits Gérés	Nombre de produits pris en compte dans le filtre courant
Mouvements (E/S)	Compteurs d'entrées et de sorties (période sélectionnée)
Alertes Critiques	Nombre d'alertes critiques non résolues

Visualisations et analyses

- **Tendance des Flux Comparatifs (Line)** : entrées vs sorties sur les 10 derniers jours.
- **Santé de l'Inventaire (Doughnut)** : rupture, critique, alerte, surstock, normal.
- **Stock Total par Produit (Bar)** : Top 15 produits par volume de stock.

Panneaux opérationnels du Dashboard

- **Derniers Mouvements** : liste des 10 derniers mouvements, affichage condensé (4) puis extension via « Voir plus ».
- **État Global des Stocks** : tableau produit/site avec stock actuel, seuil minimum, statut, quantité recommandée.
- **Action « Agir »** : redirection vers `/reapprovisionnement` pour les lignes non normales.
- **Synchronisation** : timestamp d'actualisation affiché dans l'en-tête du Dashboard.

📦 2.2 Gestion des Produits

Le module produits permet la **gestion complète du catalogue de produits** de l'entreprise :

Ce que l'utilisateur peut faire :

- **Consulter les produits** en vue **grille** (cartes visuelles avec image) ou en vue **liste** (tableau détaillé).
- **Rechercher un produit** par nom en temps réel grâce au champ de recherche instantané.
- **Filtrer par catégorie** (ex: "Électronique", "Alimentation", etc.).
- **Ajouter un nouveau produit** avec : nom, description, prix, catégorie, code-barres, et image (upload depuis le PC, stocké en Base64).
- **Scanner le code-barres** d'un produit depuis un téléphone via un QR code affiché à l'écran — le code-barres scanné est automatiquement rempli dans le formulaire grâce à SignalR.
- **Modifier un produit** existant : mettre à jour les informations, changer la catégorie, ajouter/remplacer l'image.
- **Supprimer un produit** avec confirmation.
- **Créer/supprimer des catégories** directement depuis le formulaire de produit (ajout inline).
- **Assigner un produit à des entrepôts** : Lors de la création/modification, l'utilisateur peut cocher les entrepôts où le produit doit être stocké, et des lignes de stock (quantité = 0) sont automatiquement créées.

> 🔒 **Contrôle d'accès**
>
> Seuls les utilisateurs ayant la permission `manage_products` (Admin) peuvent ajouter, modifier ou supprimer des produits. Les utilisateurs avec `view_products` (Admin, Gestionnaire, Opérateur) peuvent consulter les produits.

🔄 2.3 Mouvements de Stock (Entrées / Sorties)

Le module mouvements permet d'**enregistrer et suivre toutes les opérations d'entrée et de sortie de stock** en temps réel :

Enregistrer un mouvement :

- **Entrée de stock** — Raisons possibles : achat fournisseur, retour fournisseur, production, transfert entrant, ajustement positif, stock initial.
- **Sortie de stock** — Raisons possibles : vente, retour client, produit endommagé, périmé, transfert sortant, ajustement négatif, usage interne.
- L'utilisateur sélectionne : le **produit** (avec recherche et auto-complétion), le **site/entrepôt**, la **quantité**, la **raison**, et des **notes** optionnelles.
- Pour les sorties : sélection obligatoire d'un site de **destination**.

Validation intelligente :

- **Contrôle de stock disponible** : Lors d'une sortie, l'application vérifie que la quantité demandée ne dépasse pas le stock disponible sur le site sélectionné.
- **Contrôle du seuil maximum** : Lors d'une entrée, l'application vérifie que la quantité ajoutée ne dépasse pas la capacité restante avant le seuil maximum.
- **Filtrage des produits par stock** : En mode sortie, seuls les produits ayant du stock disponible sur le site sélectionné sont proposés.
- **Filtrage des sites** : En mode entrée, seuls les entrepôts (type "warehouse") sont affichés.

Après enregistrement :

- La **quantité de stock** est automatiquement mise à jour.
- Les **alertes** sont automatiquement recalculées (si le stock passe sous un seuil → alerte créée).
- L'utilisateur peut **scanner un code-barres avec son téléphone** (via QR code + SignalR) pour sélectionner rapidement un produit.

Consultation et filtrage :

- Historique complet de tous les mouvements avec **filtres** par : type (entrée/sortie), site, raison, date, et recherche textuelle.
- Résumé avec les **totaux** : nombre d'entrées, nombre de sorties, total des quantités.
- L'utilisateur peut consulter les détails de n'importe quel mouvement passé.
- **Ajout de raisons personnalisées** : l'utilisateur peut créer ses propres raisons d'entrée ou de sortie.

🏭 2.4 Gestion des Sites (Multi-Magasin)

L'application supporte la **gestion multi-sites** pour les entreprises ayant plusieurs entrepôts, magasins ou centres de distribution :

- **Créer un site** avec : nom, adresse, ville, code fiscal, téléphone, email, responsable, type (entrepôt, magasin, centre de distribution, production), capacité maximale, et indicateur "entrepôt principal".
- **Consulter les sites** en vue grille ou liste, avec filtrage par type et recherche.
- **Voir la capacité d'occupation** de chaque site : barre de progression colorée (vert → jaune → orange → rouge) basée sur le ratio stock actuel / capacité maximale.
- **Consulter les stocks d'un site** : Naviguer vers les stocks détaillés d'un site spécifique (`/sites/:id/stocks`).
- **Modifier ou supprimer un site** (avec confirmation).
- **Statistiques globales** : nombre total de sites, sites actifs, par type.

🔔 2.5 Système d'Alertes

Le système d'alertes **surveille automatiquement les niveaux de stock** et génère des notifications lorsque des situations anormales sont détectées :

Types d'alertes générées automatiquement :

Type	Déclencheur	Sévérité
Rupture de stock (out_of_stock)	Quantité = 0	🔴 Critique
Stock bas (low_stock)	Quantité < seuil d'alerte	🟠 Haute
Point de réapprovisionnement (reorder_point)	Quantité < seuil minimum	🟡 Moyenne
Surstock (overstock)	Quantité > seuil maximum	🔵 Basse

Ce que l'utilisateur peut faire :

- **Voir toutes les alertes** avec filtrage par : type, sévérité, date, produit, et statut (résolue ou non).
- **Consulter le détail d'une alerte** : type, message, produit concerné, site, date de création, sévérité.
- **Résoudre une alerte** : Marquer une alerte comme résolue via l'API backend.
- **Supprimer une alerte** avec confirmation.
- **Marquer toutes les alertes comme lues** en un clic.
- **Dédoublonnage automatique** : Le backend utilise un "Fingerprint" unique par type et stock pour éviter les alertes en double.
- **Statistiques d'alertes** : Nombre total, critiques, actives, par sévérité.

🧮 2.6 Scanner de Codes-Barres (Multi-Plateforme)

Le scanner de l'application fonctionne dans **deux contextes complémentaires** : un mode complet de consultation/gestion sur `/scanner` et un mode relais téléphone→PC sur `/scan-relay`. L'implémentation utilise **ZXing** pour la caméra et **SignalR** pour la transmission temps réel.

Routes Scanner et Accès

Route	Usage	Accès
`/scanner`	Page scanner principale (caméra, saisie, historique, détails produit, stock par site)	`permissionGuard('scan_barcode')`
`/scan`	Session mobile avec paramètres `sessionId` + `purpose`	`scannerAccessGuard` (autorisé sans login si session active)
`/scan-relay`	Mode relais dédié aux formulaires Produits/Mouvements/Transfert	`scannerAccessGuard` (query params requis en mode non authentifié)

Modes de Scan réellement implémentés

Mode	Technologie	Comportement dans l'application
Caméra locale	`@zxing/browser` (BrowserMultiFormatReader)	Lecture continue depuis webcam PC ou téléphone, traitement anti-doublon (fenêtre 1.5s)
Saisie manuelle	Input + Enter	Fallback immédiat si caméra indisponible/refusée, même pipeline de validation/lookup
Relais téléphone→PC	SignalR Hub + QR code	Le PC crée une session, le téléphone envoie le code, le PC auto-remplit le formulaire cible

Flux Relais Téléphone → PC (Produits et Mouvements)

- Le PC génère un `sessionId` et une URL QR de type `/scan-relay?sessionId=...&purpose=...&scanOnly=1`.
- Le service `ScanSessionService` joint le groupe SignalR `scan-{sessionId}` via `JoinSession`.
- Le téléphone scanne et envoie le code avec `SendScan(sessionId, purpose, code)`.
- Le PC reçoit `ScanDetected` et applique une action selon le `purpose` :
- **PRODUCT_BARCODE** : remplit automatiquement le champ `codeBarre` du formulaire produit.

- **MOVEMENT_PRODUCT** : recherche produit par code-barres puis sélectionne le produit dans le formulaire de mouvement.
- **TRANSFER_PRODUCT** : sélectionne le produit du formulaire de transfert.
- En UI mobile, l'application ouvre directement un onglet scanner (pas de modal QR), avec gestion des blocages popup.

Capacités de la page `/scanner`

- **Lookup produit** via API `GET /api/Products/by-barcode/{code}`.
- **Extraction intelligente** : accepte un code brut ou une URL contenant `barcode/codeBarre/code/ean/upc`.
- **Affichage détaillé** : produit scanné + **stock par site** (quantité, seuil, badge stock faible/OK).
- **Actions rapides locales** : entrée/sortie avec site, raison et quantité depuis le composant scanner.
- **Historique visible** avec utilisateur, date complète et temps relatif.

Historique et Traçabilité

- **Local** : conservation de 20 scans dans `localStorage` (`scanner_scan_history_v1`).
- **Backend** : persistance via `/api/StockMovements/AddScanHistory` et lecture via `/api/StockMovements/GetScanHistory`.
- **Visibilité** : utilisateur standard = ses scans; `admin` et `gestionnaire_de_stock` = scans globaux.
- **Nettoyage** : `/api/StockMovements/ClearScanHistory` (paramètre `all=true` réservé aux profils autorisés).

Actions Post-Scan sur téléphone (mode relay-only)

- **Ajouter produit sur téléphone** : en contexte `PRODUCT_BARCODE`, le bouton dédié ouvre l'écran Produits en mode ajout avec code-barres pré-rempli.
- **PRODUCT_BARCODE** : redirection vers `/products?add=1&barcode=...`.
- **MOVEMENT_PRODUCT** : redirection vers `/movements?mode=entry|exit&barcode=...`.
- **TRANSFER_PRODUCT** : redirection vers `/movements?mode=transfer&barcode=...`.

Composants et services impliqués

Élément	Rôle
`ScannerComponent`	Caméra, saisie manuelle, lookup produit, détails stock, historique et actions rapides
`ScanSessionService`	Connexion SignalR, join session, émission/réception des scans
`QrScanModalComponent`	Génération et affichage du QR code de session
`InventoryHub` (.NET)	Gestion des groupes de session et diffusion `ScanDetected`

📋 2.7 Suggestions de Réapprovisionnement

Le module de réapprovisionnement **analyse les seuils de stock** et propose des suggestions intelligentes :

- Identification automatique des produits dont le stock est **sous le seuil minimum**.
- Calcul de la **quantité recommandée** à commander pour atteindre le niveau optimal.
- Priorisation des suggestions par **urgence** (critique, attention, à commander).
- Vue par site pour savoir **quel entrepôt** a besoin de réapprovisionnement.

👥 2.8 Gestion des Utilisateurs et des Rôles

L'application fournit un système complet de **gestion des utilisateurs** réservé aux administrateurs :

Gestion des membres (Admin uniquement) :

- **Voir tous les membres** de l'équipe avec leur rôle, statut (actif/inactif), dernier login.
- **Ajouter un nouveau membre** : Nom, prénom, email, mot de passe, et attribution d'un rôle.
- **Modifier un membre** : Changer les informations, le rôle, ou le statut.
- **Activer / Désactiver un membre** : Les membres inactifs ne peuvent plus se connecter.
- **Supprimer un membre** avec confirmation.
- **Statistiques de l'équipe** : Nombre total de membres, d'admins, de gestionnaires, d'opérateurs.

Gestion des rôles et permissions :

- **Consulter les rôles** : Admin, Gestionnaire de Stock, Opérateur — chacun avec sa couleur et ses permissions.
- **Matrice de permissions** : 17 permissions organisées en catégories (consultation, gestion, opérations), pilotées dynamiquement par le backend.
- **Attribution dynamique** : Les permissions sont vérifiées côté frontend (guards Angular) et côté backend (claims JWT).

👤 2.9 Profil Utilisateur

- Chaque utilisateur connecté peut **consulter son profil** : nom, prénom, email, rôle, dernière connexion.
- **Modifier ses informations personnelles**.
- Voir son **badge de rôle** avec la couleur correspondante (Rouge = Admin, Bleu = Gestionnaire, Vert = Opérateur).

⚙️ 2.10 Paramètres Système

- Accessible aux **administrateurs uniquement**.
- Configuration de l'application et des préférences système.

🔁 2.11 Flux de Travail Principal

Voici le flux de travail typique d'un opérateur ou gestionnaire utilisant l'application :

1️⃣ Connexion → 2️⃣ Dashboard → 3️⃣ Vérifier alertes

↓

4 Scanner produit → 5 Enregistrer mouvement → 6 Stock mis à jour

↓

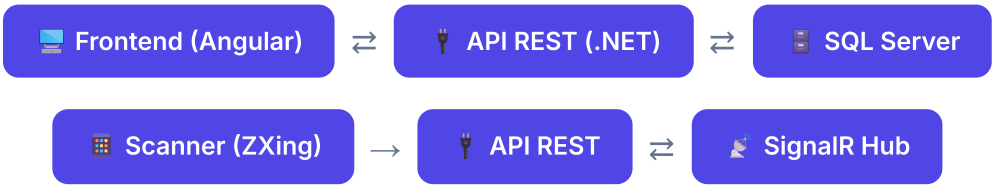
7 Alertes recalculées → 8 Réappro. suggéré → 9 Commander

🎯 En résumé, ce que fait Inventory Pro :

- ✅ Gérer le catalogue de produits avec images, codes-barres et catégories
- ✅ Suivre les stocks en temps réel sur plusieurs sites/entrepôts
- ✅ Enregistrer chaque entrée et sortie de stock avec traçabilité complète
- ✅ Alerter automatiquement quand un stock est bas, en rupture ou en surstock
- ✅ Scanner des codes-barres via caméra, téléphone distant, ou saisie manuelle
- ✅ Proposer des réapprovisionnements intelligents basés sur les seuils
- ✅ Gérer les utilisateurs avec 3 niveaux de rôles et 17 permissions
- ✅ Fournir un tableau de bord avec toutes les métriques clés
- ✅ Communiquer en temps réel entre PC et téléphone via SignalR

3. Architecture Technique

L'application suit une architecture **client-serveur** découplée avec un frontend SPA (Single Page Application) et un backend API REST.



Architecture Frontend

Pattern : Standalone Component Architecture

L'application Angular utilise des **composants autonomes** (standalone components) avec le **lazy loading** pour chaque module fonctionnel. L'état est géré par des **Angular Signals** (réactivité fine).

Architecture Backend

Pattern : Clean Architecture + CQRS

Le backend est structuré en 4 couches distinctes suivant le principe de Clean Architecture :

- **Domain** — Modèles métier, interfaces, commandes, requêtes
- **Application** — Contrôleurs API, handlers, DTOs, services, Startup
- **Data** — DbContext (EF Core), Migrations, Repositories
- **Infra** — Injection de dépendances, configuration infrastructure

4. Stack Technologique

Catégorie	Technologie	Version	Rôle
Frontend Framework	Angular	21.x	Framework SPA principal
Langage Frontend	TypeScript	5.9	Typage statique
Styling	SCSS + TailwindCSS	3.4	Styles & utilitaires CSS
UI Components	Angular Material	21.x	Composants Material Design
Scanner	ZXing (browser + library)	0.21	Lecture codes-barres/QR
QR Code	qrcode	1.5	Génération de QR codes
Temps Réel (Client)	@microsoft/signalr	10.0	Connexion WebSocket au Hub
Backend Framework	ASP.NET Core	—	API REST
ORM	Entity Framework Core	—	Accès données SQL Server
CQRS / Mediator	MediatR	—	Command/Query Separation
Mapping	AutoMapper	—	DTO ↔ Entity mapping
Auth	JWT Bearer	—	Authentification par token
API Docs	Swagger / OpenAPI	v1	Documentation interactive
Temps Réel (Serveur)	SignalR	—	Notifications temps réel
Base de données	SQL Server (Express)	—	Stockage persistant
Tests	Vitest	4.0	Tests unitaires frontend

5. Modules Fonctionnels (Frontend)

L'application est organisée en **14 modules fonctionnels** (13 protégés + 1 public), chargés dynamiquement (lazy loading) :

#	Module	Route	Description	Accès
1	**Dashboard**	`/dashboard`	Tableau de bord avec statistiques, activités récentes, top produits	Authentifié
2	**Products**	`/products`	Gestion des produits (CRUD, filtrage, vues grille/liste, images)	view_products
3	**Movements**	`/movements`	Suivi des mouvements de stock (entrée/sortie) avec historique	view_movements
4	**Sites**	`/sites`	Gestion des sites/entrepôts (localisation, capacités, responsables)	view_sites
5	**Site Stocks**	`/sites/:siteId/stocks`	Stocks détaillés d'un site spécifique	view_stocks
6	**Stocks Overview**	`/stocks`	Vue globale des stocks (tous sites)	view_stocks
7	**Alerts**	`/alerts`	Alertes système (rupture, seuil bas, surstock), résolution	view_alerts
8	**Réappro.**	`/reapprovisionnement`	Suggestions de réapprovisionnement basées sur les seuils	view_reapprovisionnement
9	**Scanner**	`/scanner`, `/scan`, `/scan-relay`	Scan ZXing + relais téléphone↔PC par session	scan_barcode / scannerAccessGuard

#	Module	Route	Description	Accès
10	**Members**	`/members`	Gestion des membres de l'équipe (rôles, statuts)	manage_users
11	**User Mgmt**	`/user-management`	Administration avancée des utilisateurs	manage_users
12	**Settings**	`/settings`	Paramètres système et permissions	manage_roles
13	**Profile**	`/profile`	Page profil utilisateur connecté	Authentifié
14	**Auth/Login**	`/auth/login`	Formulaire de connexion	Public

6. Architecture Backend (API REST)

Structure des couches

```
Backend/ ├── Domain/ # Couche Domaine │ ├── Models/ # 10 entités (Product, User, Stock, Site, Alert, Role, Permission, etc.) │ ├── Commands/ # Commandes génériques (Add, Put, Remove) │ ├── Queries/ # Requêtes génériques (Get, GetList, GetPaged, Count, Exists) │ ├── Handlers/ # Handlers génériques CQRS │ ├── Interface/ # Interfaces (IGenericRepository, IJwtService, etc.) │ └── Enums/ # Énumérations ├── Application/ # Couche Application (API) │ ├── Controllers/ # 10 contrôleurs REST │ ├── Commands/ # Commandes spécifiques (StockMovement CRUD) │ ├── Queries/ # Requêtes spécifiques (GetProductByBarcode) │ ├── Handlers/ # Handlers spécifiques │ ├── Hubs/ # SignalR Hub (InventoryHub) │ ├── Services/ # Services (JWT, Alert) │ ├── Dtos/ # Data Transfer Objects │ ├── Mapping/ # AutoMapper profiles │ ├── Events/ # Événements (StockChangedEvent) │ └── Startup.cs # Configuration DI, Auth, CORS, Swagger ├── Data/ # Couche Données │ ├── Context/ # AppDbContext (EF Core) │ ├── Migrations/ # Migrations SQL Server │ └── Repositories/ # GenericRepository<T> └── Infra/ # Couche Infrastructure └── DependencyContainer.cs
```

7. Modèle de Données

La base de données **SQL Server** (nom : `Inventaire`) contient les entités suivantes :

Entité	Champs Principaux	Relations
User	Id_u, Nom, Prenom, Email, MotDePasse, Status, RoleId, LastLogin	→ Role (N:1), → StockMovement (1:N)
Role	RoleId, Nom, Description	→ User (1:N), → RolePermission (1:N)
Permission	permissionId, Code_p, Description	→ RolePermission (1:N)
RolePermission	RoleId, PermissionId	→ Role (N:1), → Permission (N:1)
Product	id_p, Nom, Description, CodeBarre, Prix, id_c, ImageUrl	→ Category (N:1), → Stock (1:N)
Category	id_c, Nom, Description	→ Product (1:N)
Site	Id_site, Nom, Adresse, Ville, code_fiscale, Telephone, Email, ResponsableSite, Type, Capacite, EstEntrepotPrincipal	→ Stock (1:N)
Stock	id_s, QuantiteDisponible, SeuilAlerte, SeuilSecurite, SeuilMinimum, SeuilMaximum, id_p, Id_site	→ Product (N:1), → Site (N:1), → StockMovement (1:N), → Alert (1:N)
StockMovement	id_sm, DateMouvement, Raison, Quantite, Type (entry/exit), Note, Destination, Id_s, Id_u	→ Stock (N:1), → User (N:1)
Alert	Id_a, Type, Message, DateCreation, Resolue, Severity, Status, Fingerprint, ClosedAt, id_s	→ Stock (N:1)

Diagramme Relationnel (Résumé)

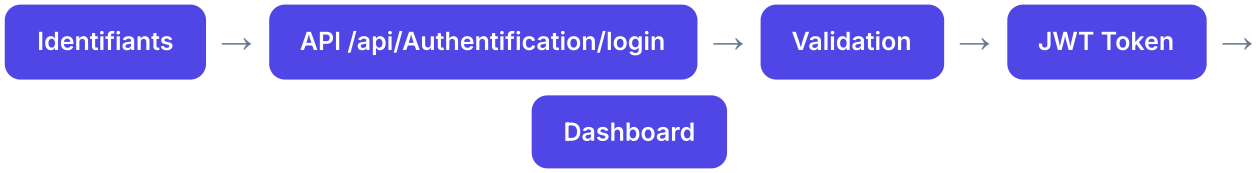
```
User ──N:1──▶ Role ◀──1:N── RolePermission ──N:1──▶ Permission Product ──N:1──▶
Category Product ──1:N──▶ Stock ──N:1──▶ Site Stock ──1:N──▶ StockMovement ──N:1──▶
User Stock ──1:N──▶ Alert
```

8. Système d'Authentification & Autorisation

Authentification (JWT)

- Le backend émet des **tokens JWT** signés avec HMAC-SHA256
- Durée de validité : **1440 minutes** (24 heures)
- Issuer : `InventaireApi` | Audience : `InventaireApp`
- Le token est stocké côté client et envoyé via le header `Authorization: Bearer <token>`

Flux d'authentification



Autorisation (RBAC)

L'application implémente un système **RBAC (Role-Based Access Control)** avec 3 rôles prédéfinis :

Rôle	Nom Backend	Couleur	Description
Admin	`admin`	Rouge (#f44336)	Accès complet au système, gestion des utilisateurs et rôles
Gestionnaire de Stock	`gestionnaire_de_stock`	Bleu (#2196f3)	Gestion des mouvements, alertes, rapports. Pas d'accès à la gestion des utilisateurs
Opérateur	`operateur`	Vert (#4caf50)	Opérations de base : scan, consultation produits, entrée/sortie

Protection des Routes (Frontend)

Les guards Angular protègent les routes et gèrent aussi l'accès scanner relay :

- `authGuard` — Vérifie l'authentification
- `adminGuard` — Restreint aux administrateurs
- `stockManagerGuard` — Restreint aux gestionnaires de stock
- `permissionGuard('permission')` — Vérifie une permission spécifique
- `scannerAccessGuard` — Autorise les sessions scanner QR sans login (query params sessionId/purpose)
- `roleGuard([...])` — Vérifie une liste de rôles autorisés

9. Matrice des Permissions

Le système définit **17 permissions canoniques** réparties en 3 catégories. Le catalogue est piloté par le backend (`/api/Roles/permissions`) et appliqué dynamiquement côté frontend + backend.

Permission	Description	Admin	Gestionnaire	Opérateur
📋 **Permissions de Consultation**				
`view_dashboard`	Voir le tableau de bord	✓	✓	✗
`view_products`	Voir les produits	✓	✓	✓
`view_movements`	Voir les mouvements	✓	✓	✗
`view_sites`	Voir les sites	✓	✓	✗
`view_stocks`	Voir les stocks	✓	✓	✗
`view_alerts`	Voir les alertes	✓	✓	✗
`view_reports`	Voir les rapports	✓	✓	✗
`view_reapprovisionnement`	Voir le réappro.	✓	✓	✓
⚙️ **Permissions de Gestion**				
`manage_products`	Gérer les produits	✓	✗	✗
`manage_movements`	Gérer les mouvements	✓	✓	✗
`manage_sites`	Gérer les sites	✓	✗	✗
`manage_stocks`	Gérer les stocks	✓	✓	✗

Permission	Description	Admin	Gestionnaire	Opérateur
`manage_alerts`	Gérer les alertes	✓	✓	✗
`manage_users`	Gérer les utilisateurs	✓	✗	✗
`manage_roles`	Gérer les rôles	✓	✗	✗
`manage_reapprovisionnement`	Gérer le réappro.	✓	✓	✗
🔧 **Permissions Opérationnelles**				
`scan_barcode`	Scanner les codes-barres	✓	✗	✓

10. Services & Logique Métier

Services Frontend (Angular)

Service	Fichier	Responsabilité
AuthService	auth.service.ts	Authentification, gestion JWT, session utilisateur
AuthorizationService	auth-authorization.service.ts	Vérification permissions, contrôle d'accès RBAC
DashboardService	dashboard.service.ts	Agrégation des KPIs et données dashboard
ProductService	product.service.ts	CRUD produits, filtrage, recherche
MovementService	movement.service.ts	Gestion mouvements de stock (entrée/sortie)
SiteService	site.service.ts	CRUD sites/entrepôts
StockService	stock.service.ts	Gestion des niveaux de stock par site
AlertService	alert.service.ts	Gestion alertes, résolution, marquage lu
CategoryService	category.service.ts	Gestion des catégories de produits
RolesService	roles.service.ts	Gestion des rôles et permissions backend
UserService	user.service.ts	CRUD utilisateurs
ScanSessionService	scan-session.service.ts	Gestion de sessions de scan codes-barres

Services Backend (.NET)

Service	Responsabilité
JwtService	Génération et validation des tokens JWT
AlertService	Moteur d'alertes : génération automatique basée sur les seuils de stock, dédoublonnage par Fingerprint

Service	Responsabilité
GenericRepository<T>	Repository générique EF Core pour toutes les entités (CRUD, pagination, comptage)

Patterns CQRS (MediatR)

Handlers Génériques

Pour chaque entité (Product, User, Stock, Site, Alert, Category, Role, StockMovement), les handlers suivants sont enregistrés :

- `AddGenericHandler<T>` — Création
- `PutGenericHandler<T>` — Mise à jour
- `GetGenericHandler<T>` — Lecture unitaire
- `GetListGenericHandler<T>` — Liste complète
- `GetPagedListGenericHandler<T>` — Liste paginée
- `RemoveGenericHandler<T>` — Suppression
- `GetCountGenericHandler<T>` — Comptage
- `ExistsGenericHandler<T>` — Vérification d'existence

11. Fonctionnalités Temps Réel (SignalR)

L'application utilise **SignalR** pour les notifications temps réel :

- **Hub Endpoint** : `/hubs/inventory`
- **Événements** : `StockChangedEvent` déclenche le handler `StockChangedEventHandler`
- **Usage** : Notification automatique quand un mouvement de stock modifie la quantité disponible, déclenchant potentiellement une alerte

12. APIs REST — Contrôleurs

Contrôleur	Base URL	Opérations
AuthentificationController	/api/Authentification	Login JWT
ProductsController	/api/products	CRUD produits, recherche par code-barre
CategoriesController	/api/categories	CRUD catégories
StocksController	/api/stocks	CRUD stocks, gestion seuils, stock par site
StockMovementsController	/api/stockmovements	Mouvements CRUD, historique filtré par rôle, scan history, raisons personnalisées
SitesController	/api/sites	CRUD sites/entrepôts
AlertsController	/api/alerts	Alertes enrichies (site/produit/utilisateur), visibilité par rôle, résolution/suppression
UsersController	/api/users	CRUD utilisateurs, changement de rôle, activation/désactivation
RolesController	/api/roles	CRUD rôles, affectation permissions
ProfileController	/api/profile	Consultation/modification profil connecté

13. Sécurité

Mesure	Détails
Authentification JWT	Tokens signés HMAC-SHA256 avec validation Issuer/Audience/Lifetime
Autorisation dynamique	Policies dynamiques `PermissionAuthorize` + `PermissionAuthorizationPolicyProvider` + `PermissionAuthorizationHandler` (permissions résolues depuis la DB)
CORS	Configuré pour autoriser le frontend Angular

Mesure	Détails
Protection XSS	Sanitization automatique Angular
Guards de routes	6 guards/factories : authGuard, adminGuard, stockManagerGuard, roleGuard, permissionGuard, scannerAccessGuard
Swagger sécurisé	Documentation API avec schéma Bearer Token
Dédoublonnage alertes	Fingerprint unique par type + stock pour éviter les doublons

14. Interface Utilisateur (UI/UX)

Design System

- **Framework UI** : Angular Material + composants personnalisés SCSS
- **Utilitaires CSS** : TailwindCSS 3.4 pour les classes utilitaires
- **Icônes** : Material Icons
- **Typographie** : Police Inter (Google Fonts)
- **Palette** : Gradient principal violet (#667eea → #764ba2)

Éléments d'interface

- **Sidebar rétractable** avec icônes, badges de notification et navigation contextuelle par rôle
- **Header** avec barre de recherche globale, notifications et menu utilisateur
- **Cartes statistiques** avec tendances et indicateurs visuels
- **Vues grille/liste** pour les produits avec filtrage avancé
- **Badges de rôle** : composant `<app-role-badge>` coloré par rôle
- **Design responsive** : Desktop, Tablette et Mobile avec menu hamburger
- **Animations** : Transitions et hover effects via Angular Animations

15. Déploiement & Configuration

Prérequis

- Node.js v18+ | npm v9+
- SQL Server Express
- SDK .NET

Configuration Frontend

```
# Installation des dépendances npm install # Démarrage du serveur de développement npm start # URL : http://localhost:4200
```

Configuration Backend

```
# Chaîne de connexion (appsettings.json)
Server=MSI\SQLEXPRESS;Database=Inventaire;Trusted_Connection=True # Swagger UI
http://localhost:<port>/swagger/index.html
```

Proxy Angular → Backend

Le fichier `proxy.conf.json` redirige les appels API du frontend vers le backend .NET.

16. Résumé des Statistiques du Projet

14
Modules Frontend

10
API Controllers

10
Entités de données

12
Services Frontend

17
Permissions

3
Rôles utilisateurs

6
Guards de routes

4
Couches Backend

Technologies Utilisées (Résumé)

Angular 21
Frontend SPA

.NET Core
Backend API

SQL Server
Base de données

SignalR
Temps réel

17. Mises à Jour Récentes et Améliorations Apportées

Cette section reflète l'état réel du code actuel (frontend + backend), avec les changements majeurs de sécurité, visibilité des données et traçabilité.

🔐 17.1 Autorisation Dynamique (Backend + Frontend)

- **Backend** : Remplacement des règles statiques par des policies dynamiques via `PermissionAuthorize`, `PermissionAuthorizationPolicyProvider` et `PermissionAuthorizationHandler`.
- **Source de vérité** : Les permissions sont résolues depuis la base de données (rôle → rolepermission → permission).
- **Frontend backend-mode** : Les permissions proviennent des rôles chargés via `/api/Roles` et `/api/Roles/permissions`, sans fallback statique de matrice.
- **Catalogue canonique** : 17 permissions incluant `view_stocks` et `manage_stocks`; suppression de `basic_entry_exit`.

📜 17.2 Visibilité Historique & Alertes par Identité

- **Historique mouvements** : un utilisateur standard voit ses propres mouvements; `admin` et `gestionnaire_de_stock` voient tout.
- **Historique scanner** : même règle de visibilité, avec traçabilité `createdByUserId` et `createdByUserName`.
- **Alertes** : `admin` et `gestionnaire_de_stock` voient toutes les alertes; les autres voient uniquement leurs alertes contextualisées.
- **Enrichissement UI** : chaque alerte expose `utilisateurNom` (aligné avec l'historique des mouvements).

🧩 17.3 Alignement Routes, Permissions et UX

- **Routes sécurisées** : alignement des guards avec les permissions `view_*` / `manage_*`.
- **Connexion robuste** : redirection post-login vers la première route autorisée pour éviter les boucles de navigation.
- **Scanner mobile** : accès relay via `scannerAccessGuard` (session QR) sans compromettre les routes authentifiées.
- **UI Stocks** : correctifs de superposition modal/overlay (seuils) pour garantir l'affichage au-dessus du layout.

📦 17.4 Persistance Métier Opérationnelle

- **Raisons de mouvements personnalisées** persistées dans `Backend/Application/App_Data/movement-reasons.json`.
- **Historique scanner** persisté dans `Backend/Application/App_Data/scan-history.json` (avec normalisation et limite de rétention).
- **Moteur d'alertes** : recalcul automatique sur changement de stock + dédoublonnage par `Fingerprint`.

✅ 17.5 État Technique Vérifié

- **Frontend** : build OK (`npm run build`) avec warnings non bloquants (budgets et dépendances CommonJS liées au QR).
- **Backend** : build OK sur `netcoreapp3.1` avec warnings de maintenance attendus (framework en fin de support).
- **Architecture** : frontend Angular standalone + signals; backend Clean/CQRS + MediatR + EF Core + JWT + SignalR.